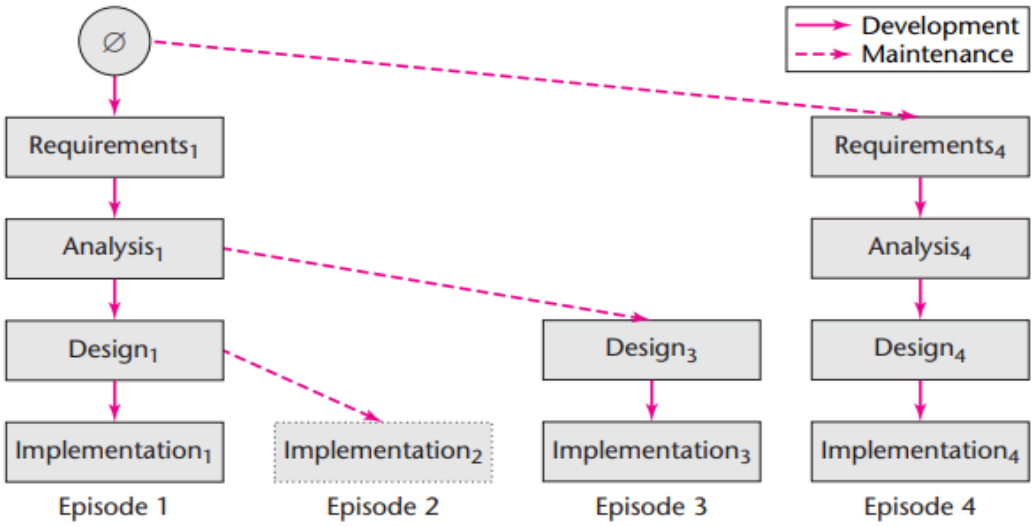
Software development lifecycle models

Software development models are structured frameworks that guide the planning, execution, and delivery of software projects. They define the sequence of development phases, such as requirements, design, programming, testing, and deployment.

Winburg Mini Case Study



Software development is an iterative, non-linear process.

The software development process is not a one-way flow from requirements to implementation. The Winburg Case Study shows that the steps Requirements → Analysis → Design → Implementation frequently need to be cycled.

- Discovering errors at a later stage (e.g., during installation) often forces the team to go back to the design or even analysis phase to fix the problem at its root.
- This "non-linear" nature means that stages can overlap or be repeated multiple times until the desired level of completion is achieved.

The stages may be subject to change during the development process.

The Winburg Case Study demonstrates that system requirements are never static.

- **Change of requirements:**When customers change their minds or the business environment fluctuates, all subsequent steps (Analysis, Design, Implementation) must be updated accordingly.
- **Environmental dependence:**Changes in hardware or device upgrades also require software adjustments to maintain compatibility and performance.

- **Consequences:**Errors in initial technique selection or design, if not detected early, will directly affect the final quality, forcing the project to be "reworked," leading to increased time and effort.

Challenges related to schedule and cost.

One of the biggest lessons from this case study is the explosion of budget and time.

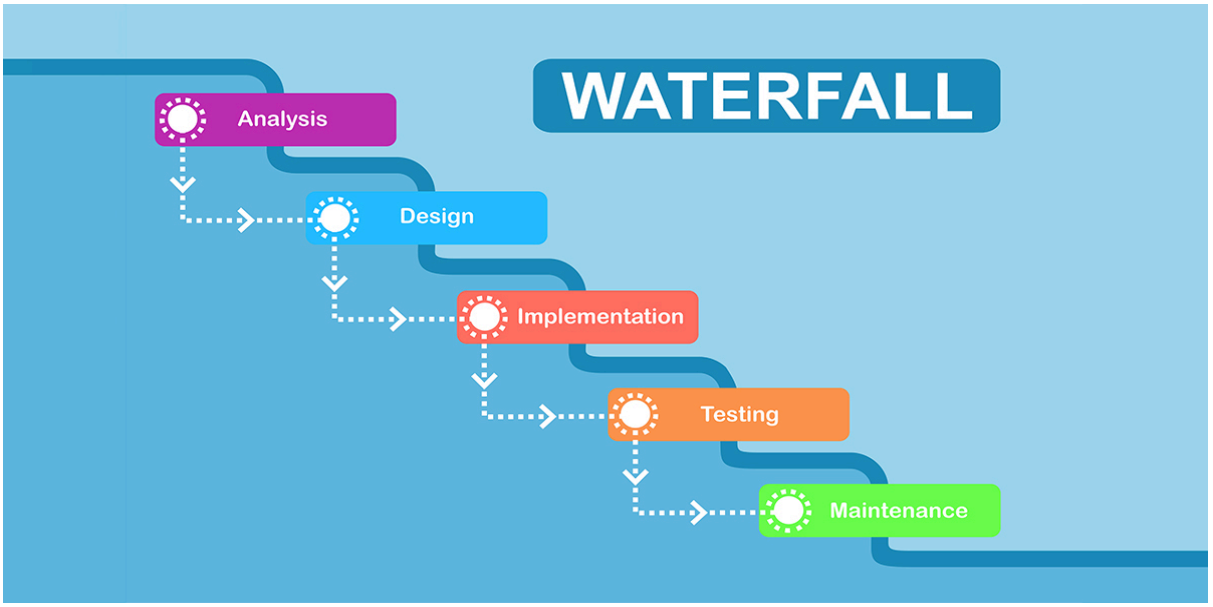
- Because of the constant revisions and changes, software projects often struggle to meet deadlines and budgets.
- Each time a step is "reversed," the opportunity cost increases and the project lifecycle lengthens.

Lifecycle model: Development and Maintenance

Stephen R. Schach emphasized that the life-cycle model must encompass both main phases:

- **Development:**Focus on building the product step-by-step process from the initial stages to the final product.
- **Maintenance:**This is actually the longest and most expensive phase. In fact, the line between development and maintenance is often blurred because software is constantly evolving to adapt to new requirements.

Waterfall model



The waterfall model is a sequential development model that allows for backtracking to previous steps to correct errors, but it is less flexible in

representing changes over time; each completed phase forms a baseline for managing the system.

Feedback Loop and Backtracking Mechanism

Although the Waterfall model is a linear model (following a top-down sequence), in practice, it allows for backtracking to address emerging issues.

- Error handling process: If an error is discovered during the Design phase, but the root cause lies in the Requirements phase, the model allows returning to the Requirements step for modification.
- Chain reaction: After modifications at the initial stage, intermediate stages such as Analysis and Design must also be updated accordingly to ensure consistency.

Distinguish between Development and Maintenance flows.

In Schach's Waterfall model diagram, the difference between these two activities is clearly shown through line symbols:

- Solid arrow: Indicates the main development flow, moving from one stage to the next in chronological order.
- Dashed arrow: Indicates maintenance or troubleshooting, representing feedback loops where it's necessary to go back to previous steps for adjustments.

Benefit:

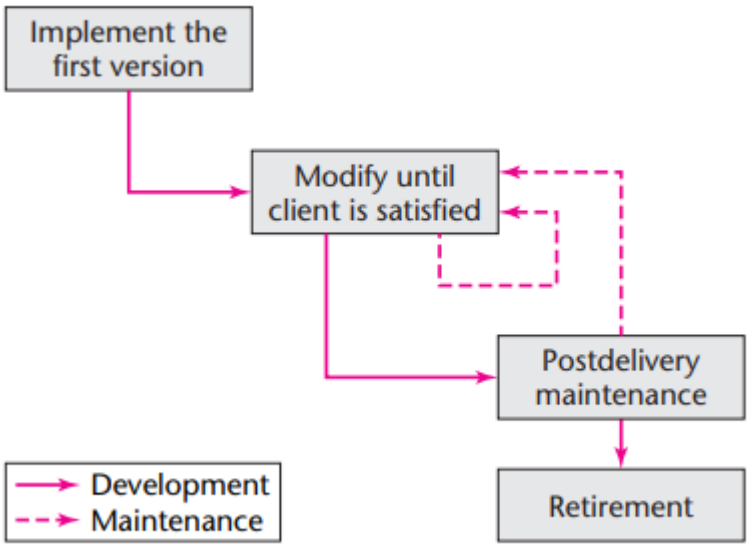
- Progress tracking: Helps managers know exactly where the project is.
- Change management: Create a fixed point of reference to compare against when new editing requirements arise.
- Version comparison: Easily identify the differences between the various stages of product development.

Limitations of the model

Despite improvements, the waterfall model still has inherent drawbacks:

- Lack of real-time synchronization: It does not clearly represent the sequence of events that change continuously over time, unlike evolutionary models.
- Inflexible: Compared to the Evolution Tree model or modern Agile methodologies, Waterfall is less adaptable to projects requiring rapid and sudden changes.

Code-and-Fix Life-Cycle Model



Below is the content about the model.**Build and Fix (Code-and-fix model)**It is divided into clearly defined sections to make it easy for you to follow and incorporate into your documents:

The nature of the Code-and-fix model

The Build-and-Fix model is considered the most rudimentary software development method, where the workflow is almost entirely stripped of foundational stages. In this approach, the development team does not define requirements, there is no analysis/specification step, and system design is completely skipped. Instead, programmers immediately begin writing source code and make continuous modifications until the program works as intended.

Application scope and practical risks

Theoretically, this approach is only acceptable for extremely small programs or simple test code. However, it is completely unsuitable and dangerous for large projects. The actual cost of this model is often very high because detecting and fixing errors after the code has been written or deployed is many times more expensive than investing the effort to follow the correct process from the very beginning.

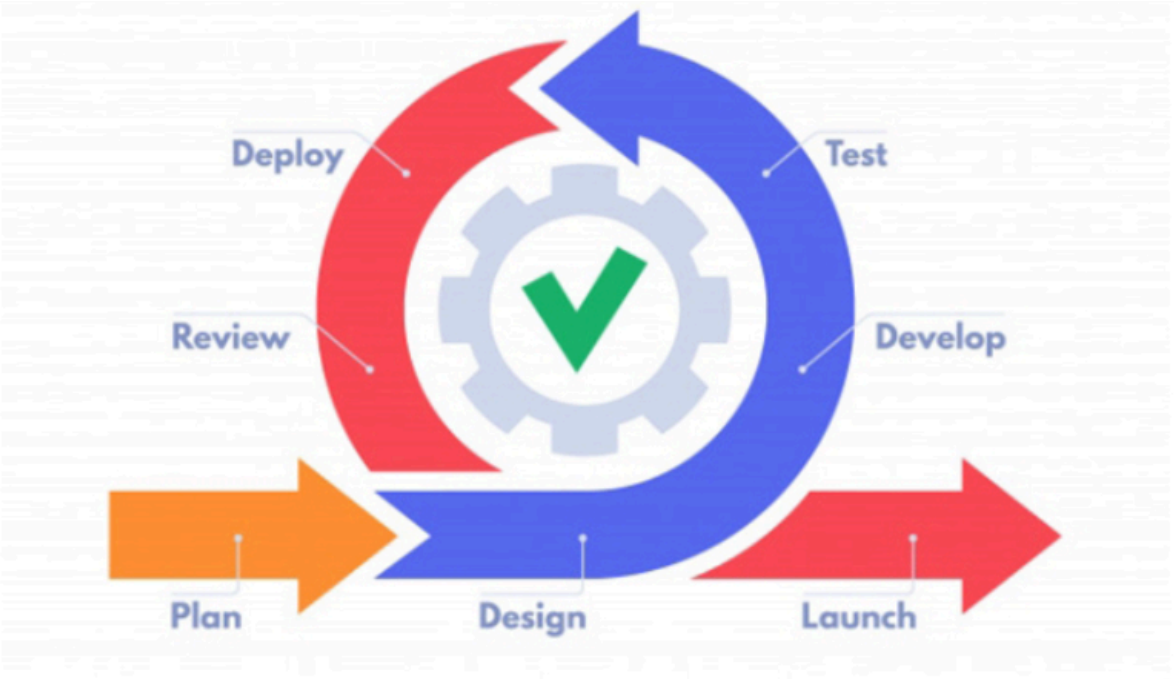
Challenges in maintenance and operation

One of the biggest weaknesses of the code-and-fix model is that it makes maintenance extremely difficult. Because the system is built without clear design or architectural documentation, subsequent takeovers will find it very hard to understand the system's logic. This inevitably leads to the easy occurrence of regression bugs—that is, fixing one bug inadvertently creating more bugs in related parts.

Causes and conclusions

In a business environment, a common cause of this misapplication often stems from management evaluating progress solely based on the number of lines of code (LOC). The pressure to have an "immediate visible product" forces programmers to skip systems thinking steps and focus on writing code. In short, this is the easiest approach to implement but yields the worst results; therefore, choosing a suitable software lifecycle model before starting a project is a mandatory requirement for a professional engineer.

Agile model



Agile is a flexible software development methodology focused on delivering products to users as quickly as possible. It is considered a significant improvement over traditional models such as Waterfall or CMMI. Agile adopts an iterative and incremental development approach, where requirements and solutions are built through close collaboration between self-managed and cross-functional teams.

Key features of the Agile model:

- Based on the iterative and incremental model, Agile breaks down the software development process into iterations. In each iteration, a part of the product is developed, tested, and refined. This allows for product improvement through each stage, rather than waiting until the entire product is complete before bringing it to market.
- Development is based on a combination of functions: Each group of functions is developed and refined individually.
- Work breakdown: In Agile, tasks are broken down into small timelines (sprints) to complete specific features before the final release.

Advantage:

- The functionalities are built quickly, clearly, and are easy to manage.
- Requirements can be easily added or changed as needed.
- The documentation is simple and easy to understand, minimizing the time and effort required for management.
- Enhance teamwork and improve the efficiency of work communication.

Disadvantages:

Despite its popularity, Agile is not suitable for all software projects:

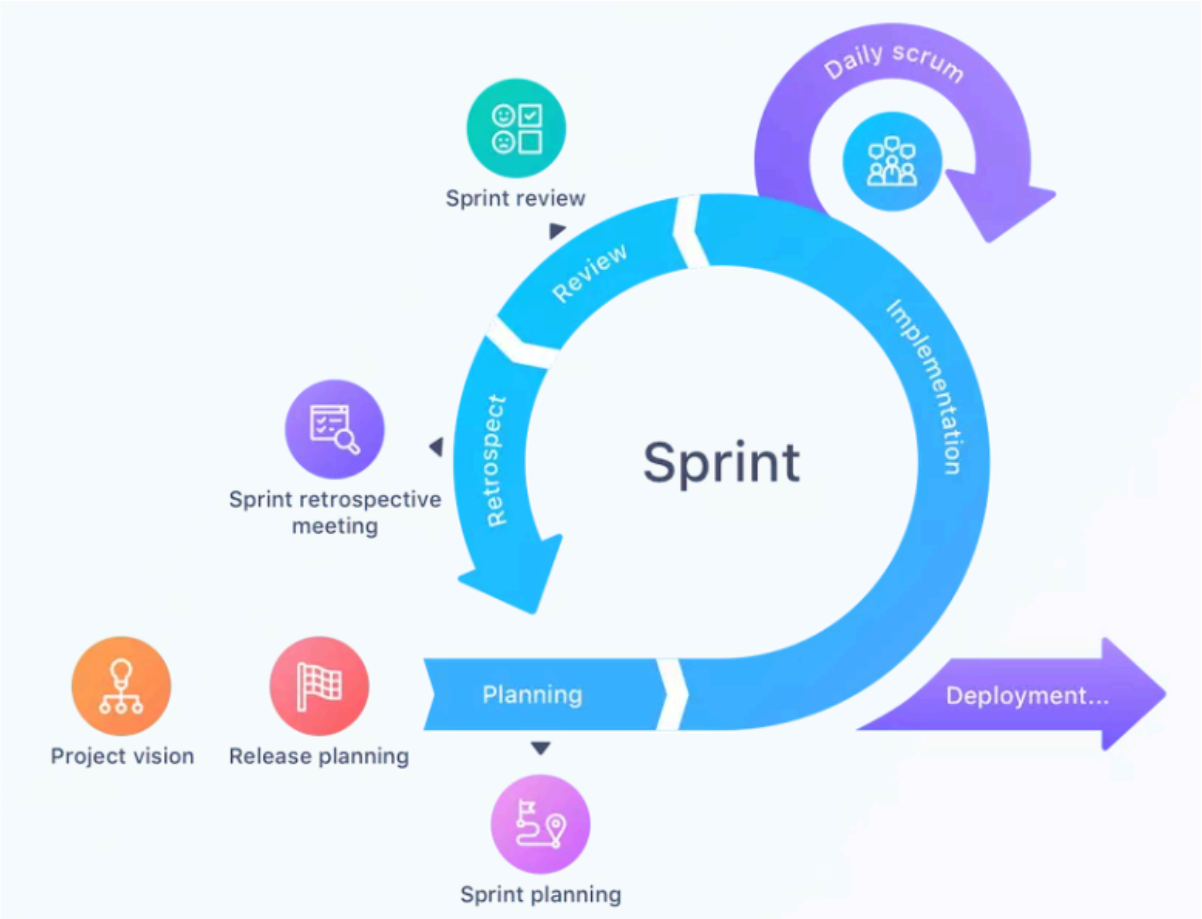
- Inefficient in handling complex dependencies.
- Requires a highly experienced team and frequent interaction with clients.
- Technology transfer or training of new members is hampered by a lack of detailed documentation.
- High risks regarding sustainability, maintainability, and scalability.

Application:

The Agile model is suitable for:

- These projects require a high level of client involvement and interaction.
- The projects need to have their functionality fully implemented within a short timeframe.
- Long-term or complex development projects, such as laboratory projects, often utilize the Agile model throughout both the development and maintenance phases.

Scrum model



The Scrum model is a flexible software development methodology that divides the process into short phases called Sprints (typically lasting from 1 to 4 weeks). Each Sprint focuses on a specific number of requirements, with the goal of completing the product after each phase. The Scrum process comprises three main elements: Organization, documentation, and processes.

In each Sprint, the team first plans and selects the requirements to be implemented, then proceeds with coding and testing to ensure the product is complete, runnable, and ready for demo.

The Scrum model requires daily meetings (Daily Scrum) in which members briefly report on progress, plans, and challenges encountered (typically lasting 15-20 minutes). Scrum focuses on meeting customer needs through flexible and rapid development, enabling continuous product adjustments and improvements throughout the development process.

The key elements of the Scrum process:

Organization:

- Product Owner: The person who owns the product and is responsible for defining and managing the Product Backlog (the list of features to be developed).
- Scrum Master: The facilitator who ensures that Scrum processes are implemented correctly and effectively.
- Development Team: A development team (4-7 members) responsible for developing, testing, and refining the product.

Artifacts:

- Product Backlog: A list of all the features that need to be developed for the product.
- Sprint Backlog: A list of features to be developed within a specific Sprint.
- Estimation: The estimated work performed by the team.

Process:

- Sprint Planning Meeting: Planning and scheduling for each Sprint.
- Daily Scrum Meeting: A short daily meeting to update progress and address challenges.
- Sprint Review: Summarizing, evaluating results, and presenting the final product of the Sprint.

Scrum enables continuous and flexible product development, easily adapting to customer feedback throughout the process.

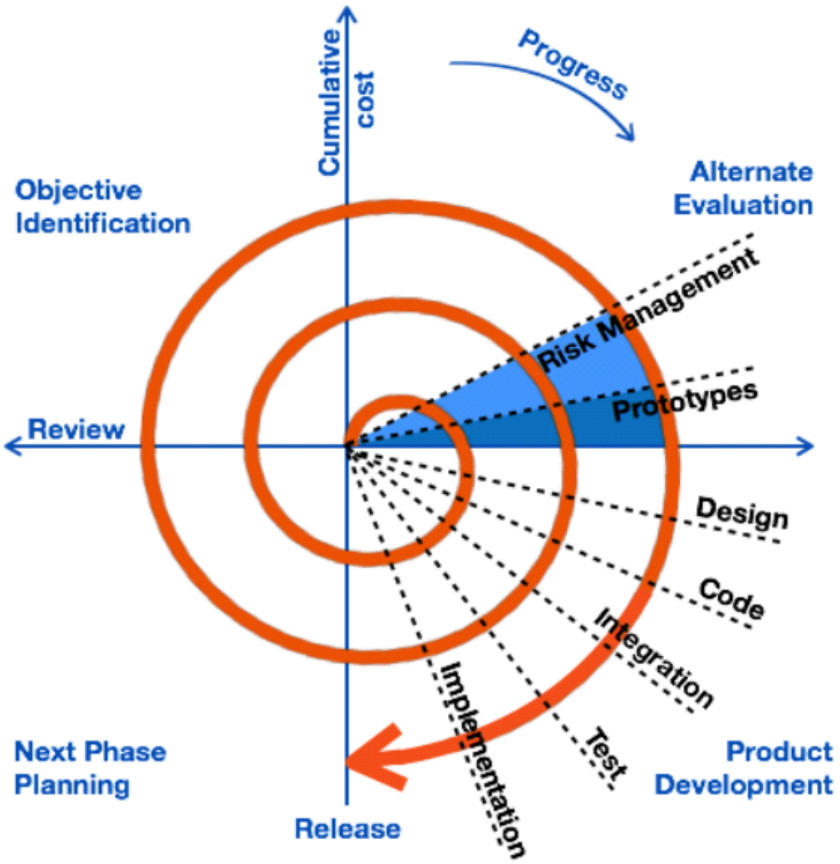
Advantage:

- Early error detection.
- Flexible to meet constantly changing customer requirements.
- Suitable for projects with unclear initial requirements.
- Developing with a customer-centric approach helps to improve customer satisfaction.

Disadvantages:

- The team needs to have the necessary skills and experience.
- The time and budget are difficult to define precisely.
- The role of the Product Owner is crucial; if not performed well, it will affect the entire project.

Spiral model



The Spiral model combines elements of the Waterfall and Prototyping models, focusing on risk control in the software development process. This model is often applied to large, complex projects with high cost and time requirements.

Model analysis:

The development process in the Spiral model is divided into iterative phases, each phase comprising the following steps:

- Objective Identification: Define specific objectives for each phase, including technical requirements and business goals.
- Alternative Evaluation: Assessing potential risks in the project and proposing measures to mitigate those risks.
- Product Development:
  Use an appropriate development model to create the system's components or features.
- Next Phase Planning: Evaluate the results of the current phase and create a detailed plan for the next phases.

Advantage:

- Optimize risk control at each stage.
- This allows for the early detection and resolution of critical issues.
- Suitable for large projects requiring high quality and reliability.

Disadvantages:

- The project management team needs to be highly skilled.
- The development costs and time are greater compared to other models.
- Not suitable for small or low-risk projects.
- It can lead to an infinite loop if change requests are not properly managed.

Application:

The spiral model is commonly applied in:

- These systems are large, complex, and require a high level of risk control.
- Projects often have multiple distinct phases or segments.

Iterative model

The iterative approach focuses on software development through repeated iterations from start to finish, meeting all requirements. After each iteration, the software is improved and a new version is created, allowing for gradual adjustments and refinements based on customer feedback.

The main characteristics of this model are:

- Instead of developing complete software based on initial specifications, this model allows for continuous testing and adjustments to achieve the optimal product.
- Each iteration produces a usable version of the software, making it easy for customers to evaluate and provide feedback.

Advantage:

- This allows for the development and refinement of the product in clear, step-by-step processes.
- Reduce the documentation time compared to the design and development time.
- Some features can be completed early, allowing customers to evaluate and provide feedback in a timely manner.
- Risk management and cost reduction are easier when scope or requirements change.

- Software is produced early in the project lifecycle, allowing customers to participate in the development process.

Disadvantages:

- It requires more resources than traditional models.
- Design or architectural issues can arise at any time, requiring immediate attention.
- Project management becomes more complex, especially when dealing with repetitive loops.
- Project progress depends heavily on the risk analysis and management phase.

Application:

The loop approach model is suitable for:

- Projects have core requirements that have been identified but still need adjustments or functional additions during the development process.
- Projects that utilize new technologies require the development team to learn and become familiar with them during implementation.
- Suitable for large and important projects requiring flexible management and adjustment capabilities.

Incremental model

The growth model divides the software development process into smaller, more manageable parts. Each part (module) goes through the entire development lifecycle, including requirements gathering, design, implementation, testing, and deployment.

The key feature of this model is that the product is not fully developed from the start but is built and released in stages. This allows customers to use and provide feedback earlier, helping to make adjustments and improvements over time.

Advantage:

- Rapid development: Each module is developed separately, which shortens the time to the first version.
- Flexibility: The model can be easily adjusted when there are changes in scope or requirements.
- Easy to test and debug: Developing in small modules makes it easy to test, detect, and fix errors before integrating them into the complete system.

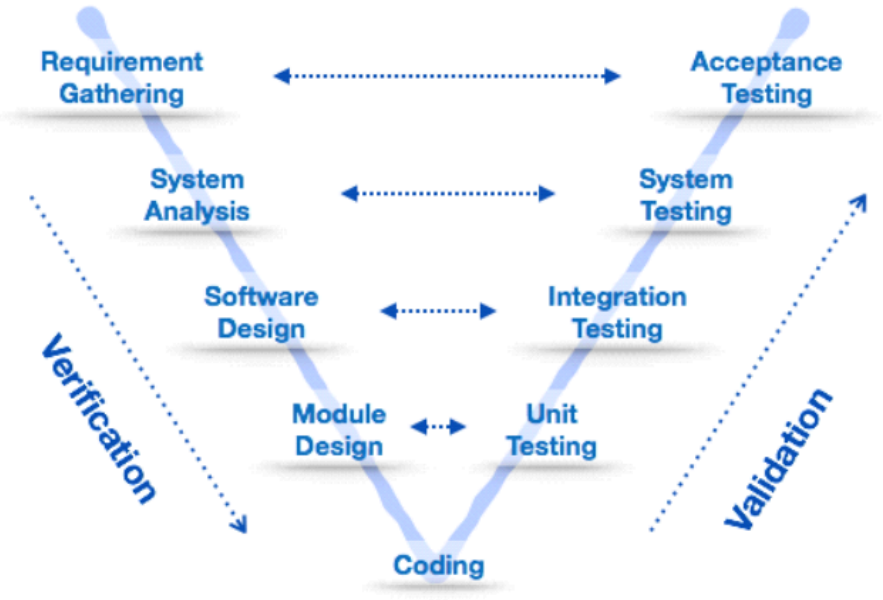
Disadvantages:

- Requires good planning and design: To ensure consistency and effectiveness, detailed planning and a well-structured system design are essential from the outset.
- Higher cost compared to the waterfall model: Total cost may increase due to the need for multiple testing and integration iterations.

Application:

- This applies to projects with clear and detailed requirements that have been described and defined.
- Suitable when customers need the product soon for use or evaluation.

V-shaped model



The V-model is an extension of the waterfall model, where each phase of software development is coupled with a corresponding testing phase. This is a highly disciplined model, ensuring that each phase only begins after the previous one has been completed.

A key feature of the V-model is the integration of testing from the outset, which helps detect and resolve issues earlier, thereby improving product quality.

Advantage:

- The development process follows strict, highly disciplined stages, making it easier to manage.

- Effective for small projects, works well when requirements are clearly defined and change infrequently.
- Easy to understand and simple, the stages are clearly defined, making it easy for the team to work according to the plan.

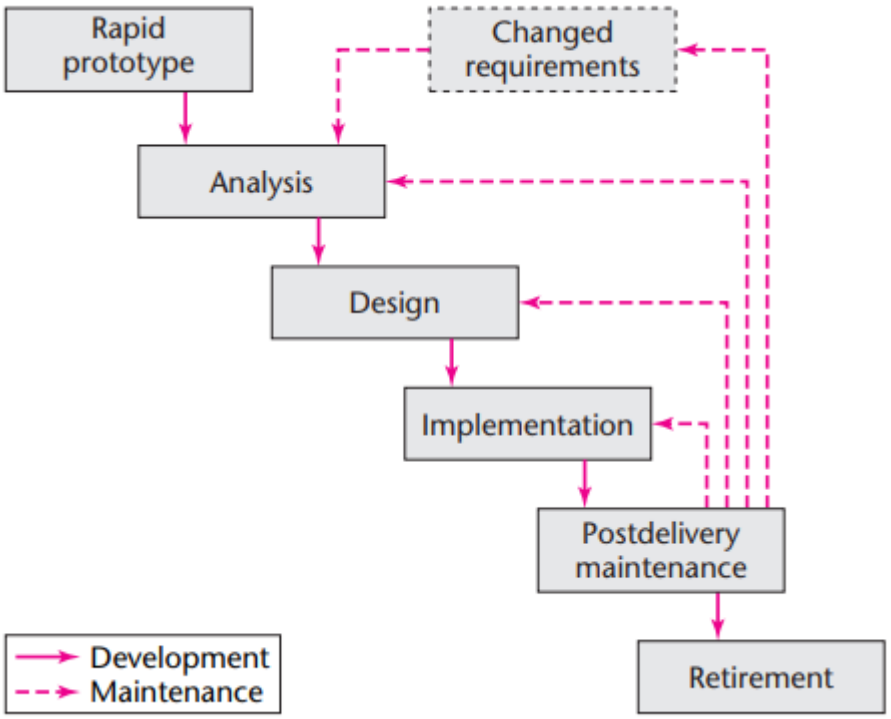
Disadvantages:

- Difficulty in risk control: Lack of flexible mechanisms to handle unexpected risks or changes.
- Not suitable for complex projects: The model is ineffective for large, long-term projects, or those requiring frequent changes.
- High risk cost: When errors occur in the early stages, correcting them later can be much more expensive.

Application:

- Suitable for projects with clear and stable requirements.
- Suitable for products where the technology is well understood by the development team.
- Suitable for short-term projects or those with minimal changes in requirements.

RAD model (Rapid Application Development)



The Rapid Application Development (RAD) model focuses on rapid software development through a prototyping process. Functionality is developed in parallel as prototypes and then integrated into a complete product. This model optimizes reusability by ensuring that prototypes and developed components can be reused in the future, saving time and costs for subsequent projects.

Advantage:

- Reduce product development time.
- Increase the reusability of software components.
- Support for quick customer reviews and feedback.

Disadvantages:

- Requires a highly skilled development team.
- Suitable only for projects with a modular structure.

Application:

- Suitable for projects with a clear modular structure.
- This is applicable when you need to create a system that requires customer changes within a short period (2-3 months).
- It requires an experienced design and development team willing to bear high costs.